



Salvataggio dati in JS

Il salvataggio dei dati in JavaScript permette di conservare informazioni generate o inserite dall'utente durante l'utilizzo di un'applicazione.

Normalmente, le variabili JavaScript mantengono i propri valori soltanto durante l'esecuzione del programma: quando una pagina viene ricaricata, i dati presenti in memoria vengono generalmente persi.

Per evitare questo comportamento, JavaScript mette a disposizione diversi strumenti di memorizzazione, come `localStorage` e `sessionStorage`, che consentono di conservare informazioni direttamente nel browser.



Questi strumenti sono particolarmente utili per salvare preferenze, impostazioni, informazioni inserite nei moduli o lo stato di una piccola applicazione.

La gestione della persistenza dei dati consiste nella capacità di mantenere disponibili determinate informazioni anche dopo la conclusione dell'esecuzione del codice.

In JavaScript possiamo distinguere il salvataggio temporaneo, effettuato attraverso variabili e strutture dati in memoria, dal salvataggio persistente, ottenuto attraverso strumenti come localStorage.

Quest'ultimo memorizza i dati sotto forma di coppie chiave-valore, utilizzando stringhe, e permette di recuperarli anche dopo la chiusura e la riapertura del browser.

Per memorizzare oggetti e array possiamo utilizzare JSON.stringify() e JSON.parse(), che consentono di convertire strutture JavaScript in stringhe e viceversa.



È importante ricordare che queste tecnologie sono pensate per dati locali non sensibili e non sostituiscono un database.

Salvataggio e recupero dei dati con localStorage

localStorage permette di salvare informazioni direttamente nel browser utilizzando una coppia chiave-valore.

I dati rimangono disponibili anche dopo aver ricaricato la pagina o chiuso il browser.

```
1. let nome = "Mario";
2. let eta = 25;
3.
4. // Salvataggio dei dati
5. localStorage.setItem("nomeUtente", nome);
6. localStorage.setItem("etaUtente", eta);
7.
8. // Recupero dei dati salvati
9. let nomeSalvato = localStorage.getItem("nomeUtente");
10. let etaSalvata = localStorage.getItem("etaUtente");
11.
12. // Visualizzazione dei risultati
13. console.log("Nome:", nomeSalvato);
14. console.log("Età:", etaSalvata);
15.
16. // Eliminazione di un dato
17. localStorage.removeItem("etaUtente");
```



- 1. Dichiarazione delle variabili: vengono create due variabili contenenti il nome e l'età di un utente.**
- 2. `setItem()`: salva un'informazione nel browser associandola a una chiave identificativa. Il primo parametro rappresenta la chiave, mentre il secondo contiene il valore da memorizzare.**
- 3. `getItem()`: recupera un valore precedentemente salvato attraverso la sua chiave. Se la chiave non esiste, restituisce null.**
- 4. `console.log()`: mostra nella console del browser i dati recuperati, verificando che il salvataggio sia avvenuto correttamente.**
- 5. `removeItem()`: elimina un singolo dato salvato, individuandolo attraverso la sua chiave.**

Nota tecnica: `localStorage` memorizza tutti i valori come stringhe. Pertanto, il numero 25, una volta recuperato, sarà rappresentato come "25". Per convertirlo nuovamente in numero possiamo utilizzare `Number(etaSalvata)`.



Salvataggio dei dati con JavaScript e HTML

In questo esempio utilizziamo un modulo HTML per permettere all'utente di inserire il proprio nome e la propria età.

Attraverso JavaScript, i dati vengono salvati nel localStorage del browser e possono essere recuperati anche dopo aver ricaricato la pagina.



```
1. <!DOCTYPE html>
2. <html lang="it">
3. <head>
4.   <meta charset="UTF-8">
5.   <title>Salvataggio dati</title>
6. </head>
7. <body>
8.
9.   <h2>Gestione Utente</h2>
10.
11.   <input type="text" id="nome" placeholder="Inserisci nome">
12.   <input type="number" id="eta" placeholder="Inserisci età">
13.
14.   <button onclick="salvaDati()">Salva</button>
15.   <button onclick="mostraDati()">Mostra</button>
16.   <button onclick="eliminaDati()">Elimina</button>
17.
18.   <p id="risultato"></p>
19.
20.   <script>
21.     function salvaDati() {
22.       let nome = document.getElementById("nome").value;
23.       let eta = document.getElementById("eta").value;
24.
25.       localStorage.setItem("nomeUtente", nome);
26.       localStorage.setItem("etaUtente", eta);
27.
28.       document.getElementById("risultato").innerText =
29.         "Dati salvati correttamente!";
30.     }
31.
32.     function mostraDati() {
33.       let nome = localStorage.getItem("nomeUtente");
34.       let eta = localStorage.getItem("etaUtente");
35.
36.       if (nome !== null && eta !== null) {
37.         document.getElementById("risultato").innerText =
38.           "Nome: " + nome + " - Età: " + eta;
39.       } else {
40.         document.getElementById("risultato").innerText =
41.           "Nessun dato salvato";
42.       }
43.     }
44.
45.     function eliminaDati() {
46.       localStorage.removeItem("nomeUtente");
47.       localStorage.removeItem("etaUtente");
48.
49.       document.getElementById("risultato").innerText =
50.         "Dati eliminati!";
51.     }
52.   </script>
53.
54. </body>
55. </html>
```

Spiegazione del codice

1. **Elementi HTML:** vengono creati due campi input per inserire nome ed età, tre pulsanti per gestire le operazioni e un paragrafo per visualizzare i risultati.
2. **document.getElementById():** permette a JavaScript di individuare un elemento HTML attraverso il suo identificativo id. La proprietà `.value` recupera il contenuto inserito dall'utente.
3. **onclick:** collega ciascun pulsante a una funzione JavaScript che viene eseguita quando l'utente lo preme.
4. **salvaDati():** recupera i valori dagli input e utilizza `localStorage.setItem()` per memorizzarli nel browser.
5. **mostraDati():** utilizza `localStorage.getItem()` per leggere le informazioni salvate e mostrare il risultato. La condizione `if` verifica che entrambe le informazioni siano presenti.
6. **eliminaDati():** utilizza `localStorage.removeItem()` per cancellare i dati precedentemente memorizzati.
7. **innerText:** modifica il contenuto testuale del paragrafo HTML, permettendo di mostrare messaggi e risultati senza ricaricare la pagina.



Errori comuni nel salvataggio dei dati con JavaScript

Gestione errata dei tipi di dato e delle chiavi: uno degli errori più frequenti nell'utilizzo di localStorage riguarda il salvataggio e il recupero delle informazioni.

I dati vengono sempre memorizzati come stringhe, quindi numeri, oggetti e array devono essere gestiti correttamente.

Per gli oggetti è necessario utilizzare `JSON.stringify()` durante il salvataggio e `JSON.parse()` durante il recupero.

Un altro errore comune consiste nell'utilizzare chiavi differenti tra `setItem()` e `getItem()`, ottenendo così un valore null.

 **È quindi fondamentale mantenere una nomenclatura coerente e verificare l'esistenza dei dati prima di utilizzarli.**

Problemi di integrazione con HTML e persistenza:

quando JavaScript interagisce con una pagina HTML, possono verificarsi errori legati agli identificativi degli elementi o all'esecuzione delle funzioni.

Ad esempio, utilizzare un id inesistente con `document.getElementById()` restituisce null, causando un errore quando si tenta di accedere a proprietà come `.value` o `.innerText`.

Un altro errore consiste nel confondere le variabili JavaScript con i dati persistenti: modificare una variabile non aggiorna automaticamente il `localStorage`, ma è necessario richiamare `setItem()`.



Infine, è importante evitare di memorizzare password o informazioni sensibili nel browser, poiché `localStorage` non è uno spazio di archiviazione sicuro per questi dati.

Buon davante a tutti

